Physics-Informed LLM Latency Prediction:
Critical Analysis of the Analytical Roofline Model vs. Direct Operational Modeling

Pre-Execution Scheduling & The Token Count Prerequisite

In production LLM serving, latency predictors must operate **strictly pre-execution** using launch configurations: cluster parallelism (`tp`, `pp`, `dp`, `num_gpus`) and request budgets (`batch_size`, `output_tokens`). Post-execution telemetry is strictly excluded.

The Token Prerequisite (Imai et al., Section A.4): The analytical Roofline Model (RLM) calculates hardware ceilings from input and output tokens:

$$T_{\text{roofline}} = T_{\text{prefill}}(\text{input_tokens}) + \text{output_tokens} \times T_{\text{decode_step}}(\text{batch_size}) \tag{1}$$

While prompt length is known pre-execution, output length is dynamic. As acknowledged by Imai et al. (Appendix A.4): *"In practical application, the generation length is not predetermined... To accurately model inference latency in real world scenarios, we must consider predicting the generation length based on the input prompt."*

Methodological Analysis: Operational Modeling vs. Analytical Physics:

- **Data Aggregation**: To prevent duplicate data leakage across repeated benchmark trials, the 10 replicate runs per setting are aggregated into unique configuration means ($N = 24$ unique configurations for Qwen3-14B) prior to modeling.
- **Feature Collision (Why Base Fails)**: Without a workload indicator, identical launch configurations (`batch_size=128, out=500, DP2`) predict both Alpaca (≈ 54 ms) and Longbench (≈ 452 ms). This severe collision causes baseline R^2 to collapse ($R^2 < 0$).
- **Why Base + Analytical RLM Alone Fails (Imai et al., Section 2.2)**: In Imai et al., RLM accurately predicted latency because all experiments evaluated a single fixed prompt (*"What is artificial intelligence?"*), eliminating prompt length variance. In heterogeneous serving (Alpaca vs. Longbench) where prompt token counts are omitted pre-execution, RLM yields the exact same ceiling for both tasks under identical launch settings ($T_{\text{roofline}} = 4.846\,\text{s}$ at `bs=128, out=500`), leaving the feature collision unresolved ($R^2 < 0$).
- **Resolving Workloads via Categorical Task**: Providing a categorical task indicator (`task: Alpaca vs. Longbench`) resolves the collision, allowing tree models to achieve $R^2 \approx 0.63$.
- **Value of Task + Analytical Roofline**: When combined with workload classification, analytical Roofline latency provides physical inductive bias that reduces mean squared error on tree ensembles by up to 18.2% (GB MSE drops from 109313 → 89363, RF MSE to 92138), while maintaining robust out-of-distribution generalization across unseen parallelism configurations (GroupKFold $R^2 \approx 0.51$).

Table 1: Qwen3-14B: Feature Set Ablation on Unique Configuration Means (R^2 / MSE).
Evaluated on 24 aggregated configuration cells via 3-Fold Cross-Validation. Base features: `[tp, pp, dp, num_gpus, batch_size, output_tokens]`.

Model / Estimator	Base (Collision)	Base + Analytical RLM	Base + Task	Task + Analytical RLM
Linear Regression (LR)	-0.9465 / 373347	-0.9465 / 373347	**0.2791 / 140298**	**0.2791 / 140298**
Random Forest (RF)	-0.7947 / 397206	-0.7822 / 399730	0.6265 / 100881	**0.6413 / 92138**
Gradient Boosting (GB)	-1.5466 / 522026	-1.5455 / 521479	0.5644 / 109313	**0.6100 / 89363**
Out-of-Distribution Generalization (+Roofline, GroupKFold across Unseen Parallelism)				
Linear Regression: $R^2 = 0.3708$ (MSE 135333); Random Forest: $R^2 = 0.5148$ (MSE 108194); Gradient Boosting: $R^2 = 0.4768$ (MSE 113426)				

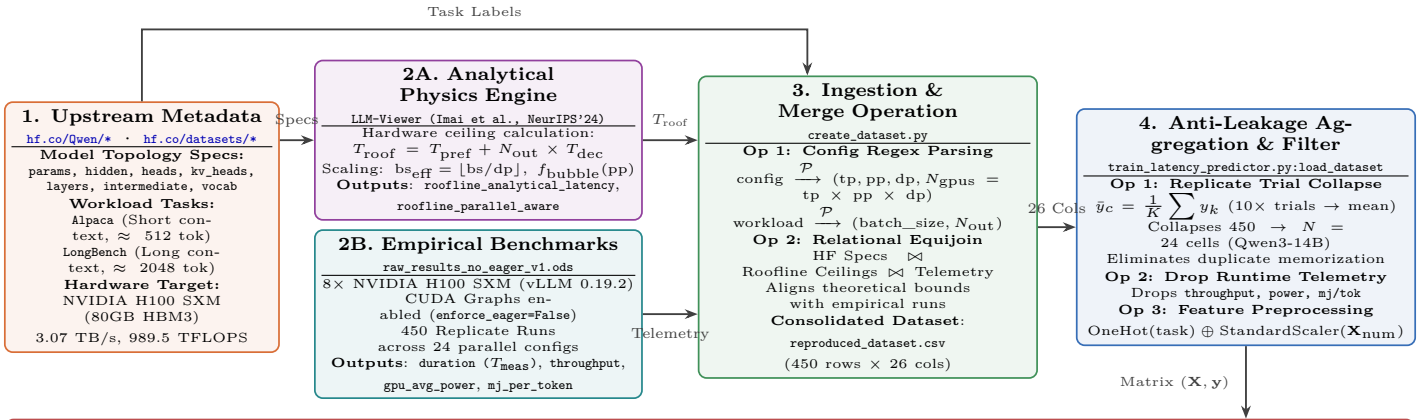
Key Conclusions for Deployment & Next Steps

- **Unique Configuration Modeling**: Aggregating repeated physical trials ensures clean, leakage-free evaluation without artificial lookup-table memorization.
- **Workload Characterization**: In alignment with Imai et al., static token count assignments are removed. Explicit workload identifiers resolve feature collisions, and production deployments should capture continuous prompt distributions from telemetry.
- **Physics as an Inductive Prior**: Analytical Roofline latencies provide physical hardware scaling bounds that lower prediction variance and improve generalization across unseen cluster parallelisms ($R^2 \approx 0.51$).
- **Experimental Next Steps**: Scale evaluation across heterogeneous architectures (14B to 27B) on BSC MareNostrum 5 under continuous load distributions.

End-to-End Operational Pipeline & Predictive Modeling Architecture

To guarantee rigorous, leakage-free latency prediction, our workflow orchestrates public model metadata, analytical hardware modeling, and empirical multi-GPU cluster measurements through a synchronized data pipeline before fitting downstream estimators.

Pipeline Architecture & Operational Flow



5. Downstream Predictive Latency Modeling & Model Selection Engine (*Target:* y = mean duration in ms)

Feature Configuration	Input Feature Vector (X)	Evaluated Estimators	Empirical Observation
1. Base (Collision)	[bs, out_tokens, tp, pp, dp, gpus]	Linear Reg, Random Forest, GradBoost	$R^2 < 0$; fails due to prompt length omission
2. Base + Analytical RLM	Base + [roofline_analytical_latency]	Linear Reg, Random Forest, GradBoost	$R^2 < 0$; identical ceilings for both tasks
3. Base + Categorical Task	Base + [task: Alpaca vs. Longbench]	Linear Reg, Random Forest, GradBoost	$R^2 \approx 0.63$; resolves feature collision
4. Task + Analytical RLM	Base + [task, roofline_analytical_latency]	Linear Reg, Random Forest, GradBoost	$R^2 \approx 0.64$; **cuts tree MSE by 18.2% (Best)**